



청약 판단을 더 쉽고 정확하게

공고문 해석부터 자격 진단, 전략 추천까지 연결하는 청약 AI 서비스

팀명 : 집없음 청년

김은진, 김지훈, 이동윤, 임준억

목차

01 프로젝트 개요

02 프로젝트 구조

03 Front UI/UX 구현

04 Django 백엔드 구현

05 배포와 운영 대응

06 시연 및 마무리

팀원 소개



김은진

PM /
FRONTEND

프로젝트 일정 / 방향 관리
React 기반 프론트엔드 UI/UX
설계



이동윤

DJANGO / 배포

Django 백엔드 API 개발
Docker 기반 컨테이너 배포



김지훈

FASTAPI / AI

FastAPI 기반 AI 엔진 연동
AI 처리 파이프라인



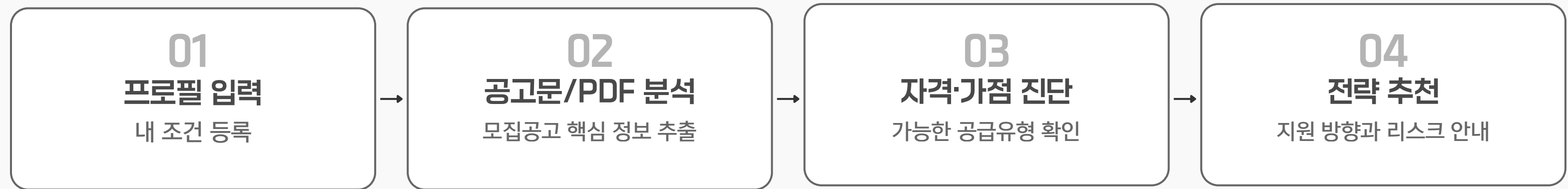
임준억

통합 / QA

전체 시스템 통합 테스트 /
품질 보증 서비스 안정성 확보

청약 판단을 더 쉽고 정확하게

공고문 해석부터 자격 진단, 전략 추천까지 한 번에 연결



공고문 정보와 사용자 조건 연결 / 청약 판단에 필요한 정보 한 번에 정리

프로젝트 목표 & 개선

AI 진단 엔진 웹 서비스 구조로 확장

#프로젝트 목표

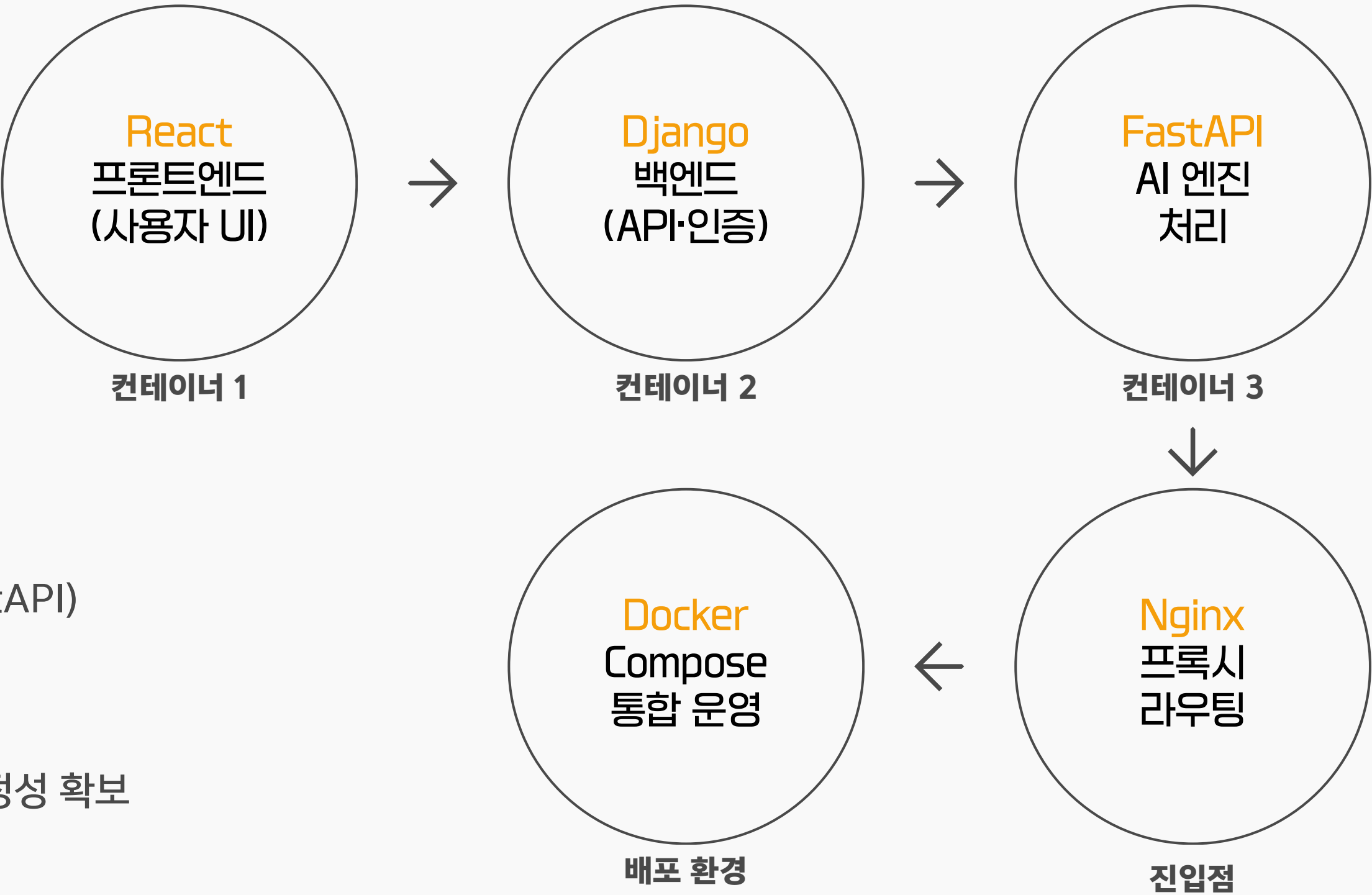
- AI 엔진 → 웹 서비스화
- React · Django · FastAPI 계층 분리
- Docker 기반 배포 구조 구축

#개선

- 사용자 인증 / 세션 관리
- 프로필 · 진단 이력 저장
- PDF 분석 · 챗봇 흐름 연결

최종 아키텍처 구조

- 3개의 독립 컨테이너(React, Django, FastAPI)
- Docker 기반 분리 운영
- Nginx 프록시 API 호출 흐름 단순화 및 안정성 확보



계층별 역할 분리



React → Django → FastAPI 단방향 호출 구조로 복잡도를 최소화

사용자 흐름 중심 UI/UX 개선

BEFORE

① 전략을 분석하고 있습니다
프로필과 공고문을 분석하고 있습니다.
보통 30~40초 정도 걸립니다. 경과 시간: 6초

AFTER

아파트 분양 모집공고 입력

아파트 입주자모집공고의 주요 내용을 붙여넣거나 PDF로 바로 분석하세요.

PDF 파일을 확인하고 있습니다

⌚ 3초 경과

C

(26.07.02.정정)2026000300 천안 아이파크 시티 1단지(2회차) 입주자모집공고문.pdf의 공고문 내용을 진단
에 사용할 수 있도록 변환합니다.

진행 상황

24%

1 파일 확인

2 내용 추출

3 결과 정리

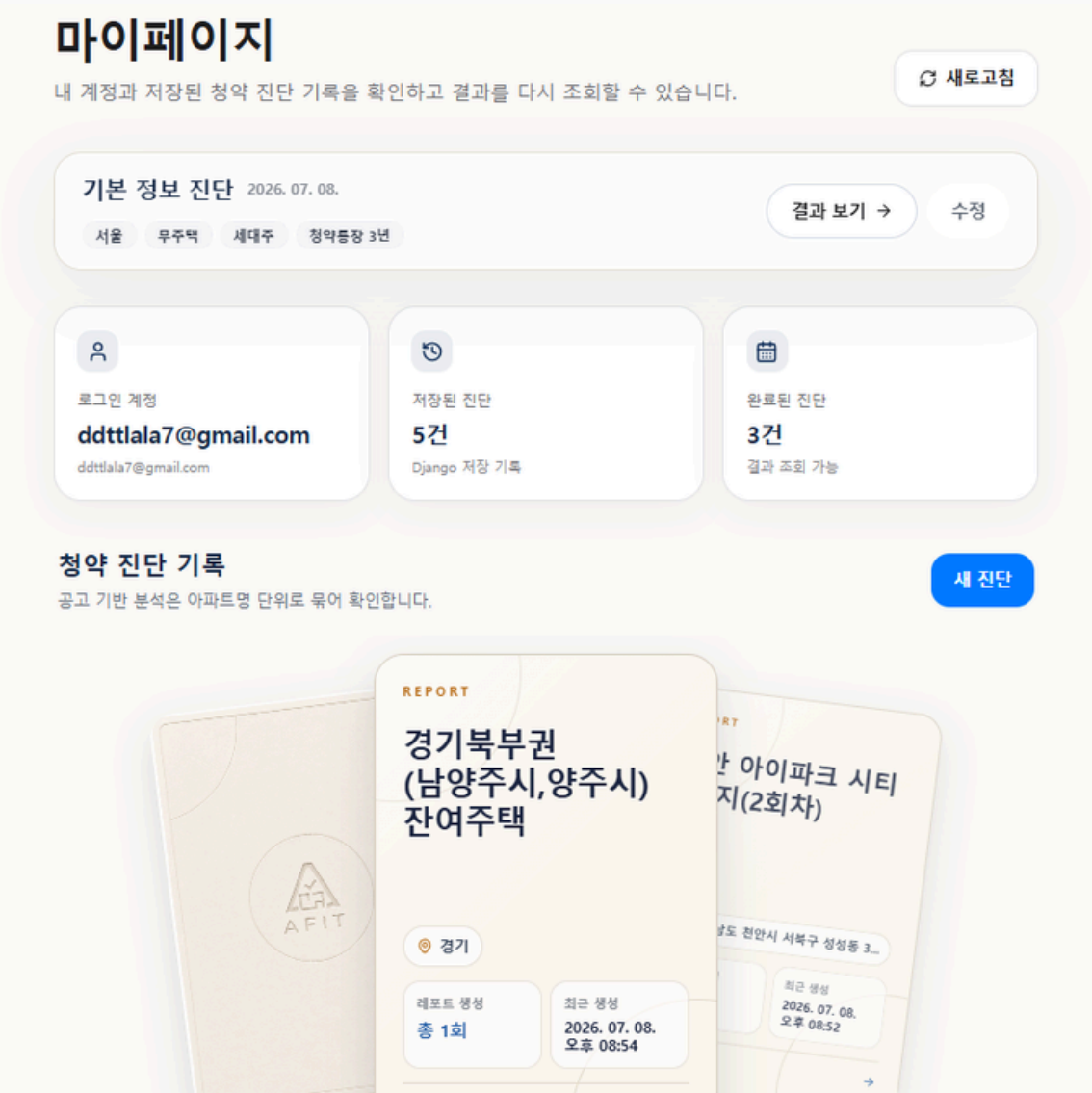
분석이 끝날 때까지 이 화면을 닫거나 새로고침하지 마세요.

사용자 흐름 중심 UI/UX 개선

BEFORE

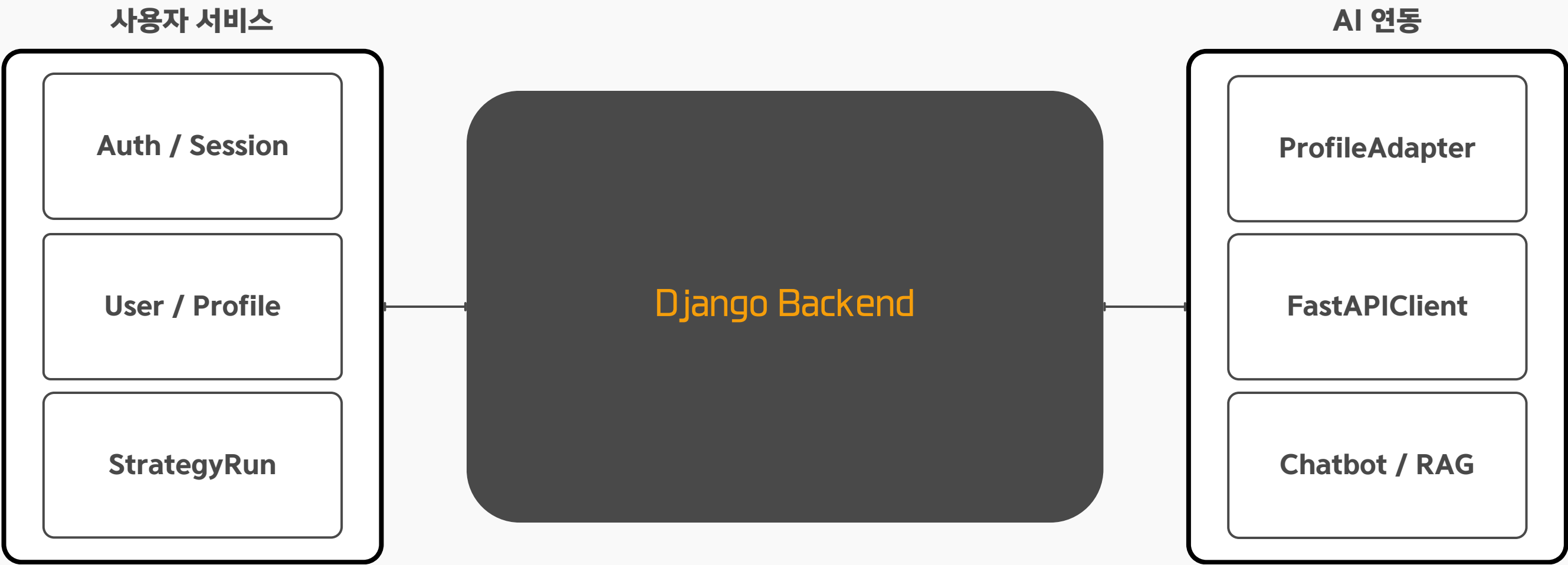


AFTER



Django 중심 통합 서비스 구조

사용자 관리와 AI 연동을 연결하는 서비스 허브



React는 Django API만 호출하고, Django가 FastAPI AI 엔진과 사용자 데이터를 연결합니다.

인증 · 세션 · 프로필 관리

Django 내장 인증과 세션을 활용한 사용자 상태 관리 구조



Django 인증 시스템으로 접근을 제어하고, Profile 모델로 사용자 맞춤 데이터를 안전하게 관리합니다.

StrategyRun 기반 진단 이력 저장

한 번의 진단 실행을 입력과 결과까지 하나의 기록 단위로 저장합니다.



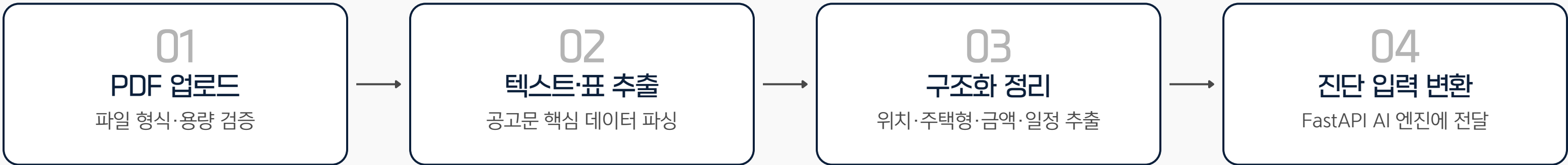
왜 필요한가

- 사용자별 진단 이력 관리
- 결과 상세 재조회
- 성공·실패 상태 추적

StrategyRun은 서비스가 **사용자의 진단 과정을 기억**하게 만드는 핵심 모델입니다.

PDF 공고문을 진단 입력으로 전환

복잡한 모집공고를 자동 추출·정리해 AI 진단에 연결합니다.



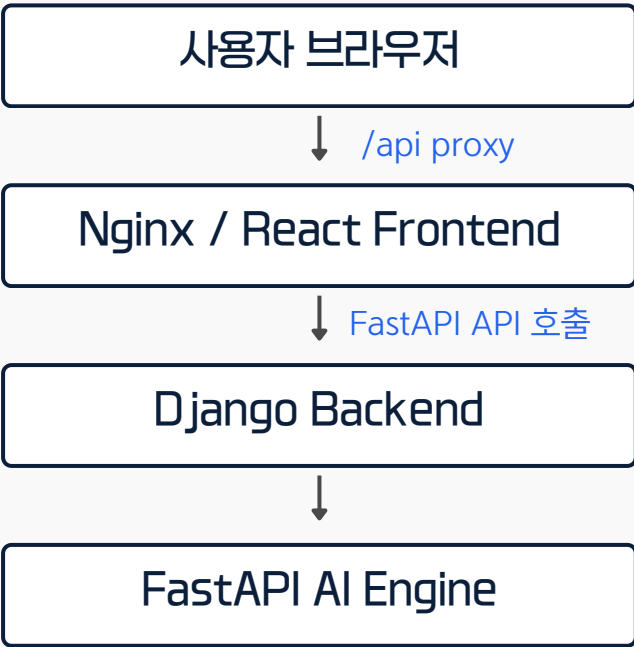
설계 포인트

- 원본 PDF 저장 없음
- 분석 결과만 StrategyRun에 기록
- 사용자 입력 부담 감소

PDF 분석은 공고문을 "읽는 문서"에서 "진단 가능한 데이터"로 바꾸는 연결 계층입니다.

Docker 기반 실행 구조

프론트·Django·FastAPI를 분리해 실행 환경을 재현 가능하게 구성했습니다.



docker-compose

frontend : 80
django-backend : 8000
fastapi-backend : 8080

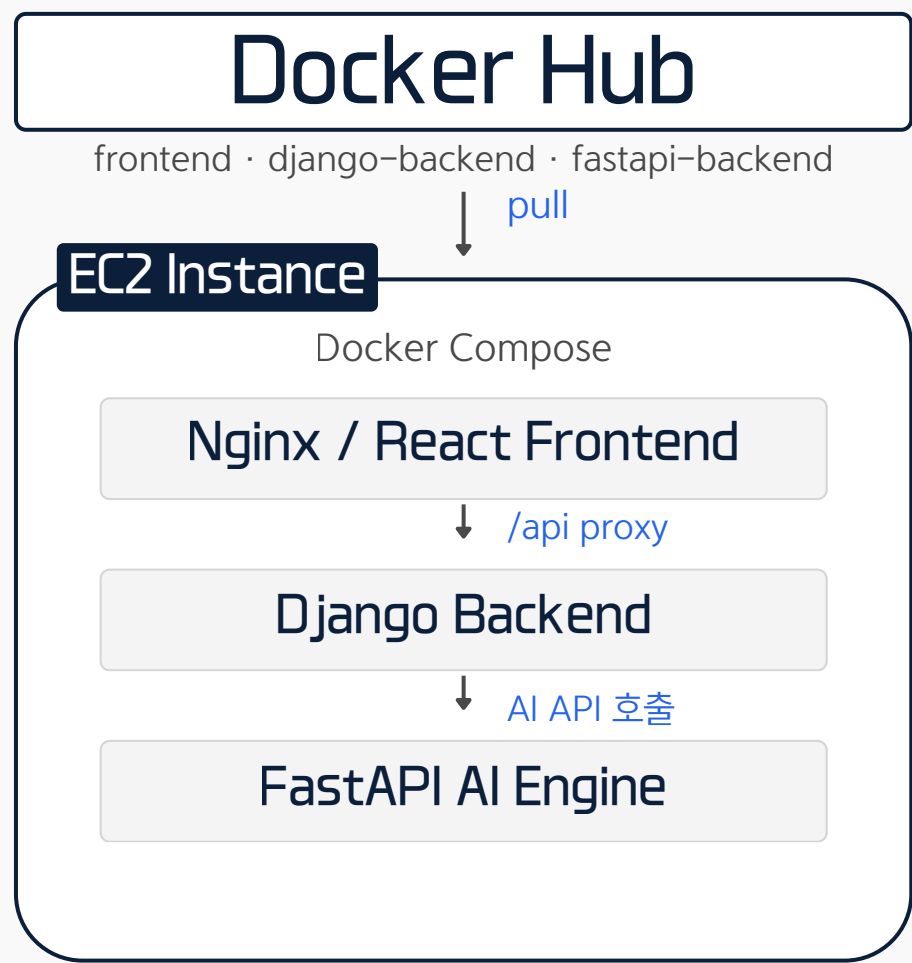
설계 포인트

- 서비스별 컨테이너 분리
- Nginx reverse proxy
- 로컬·배포 환경 재현성 확보

Docker는 세 계층을 독립적으로 실행하면서도 하나의 서비스처럼 연결하는 배포 기반입니다.

Docker 기반 EC2 배포 구조

컨테이너 단위로 서비스 분리 / EC2 재현 가능 실행 환경 구성



현재 구현

- Docker Hub 이미지 pull
- 3개 컨테이너 실행
- Nginx reverse proxy
- EC2 외부 접속 확인

후속 고도화

- PostgreSQL / RDS 전환
- S3 정적 파일 관리

Docker는 로컬 실행을 넘어, 배포 환경에서도 같은 구조로 서비스를 재현하기 위한 기반입니다.

실제 서비스 시연



감사합니다.
질문 있으시면
말씀해 주세요.